Ex. No. 6	**JAVASCRIPT - STYLES AND ANIMATIONS**
Date of Exercise	_____
GitHub Host Link	https://github.com/adhishm-0/web-technology-lab/tree/main/exp%206

Aim

To create webpages using JavaScript styles, timing events, Date objects, classes and objects, and animations.

Description

This experiment demonstrates five programs: a digital clock, a changing background color, mouse-based font resizing, a moving car animation, and student record display with birth-year classification. Runnable files are provided in the accompanying project folder.

IMPORTANT COMMANDS USED :

• setInterval() and setTimeout() — schedule repeated or delayed code

• clearInterval() and clearTimeout() — stop scheduled actions

• Date(), getHours(), getMinutes(), getSeconds() — read current time

• element.style.property — changes an element's CSS style

• addEventListener() — responds to mouse and button actions

• class, constructor(), new — define and create objects

• document.getElementById(), textContent — access and update page content

Procedure

1. Create index.html, style.css, and script.js.

2. Add one section and result area for each question.

3. Use Date and setInterval() for the clock.

4. Generate RGB values and update the background every 30 seconds.

5. Use mouseover and mouseout to resize the paragraph.

6. Move the car with setTimeout() and stop it on button click.

7. Create Student objects and categorize students by birth year.

8. Open the page in a browser and verify each output.

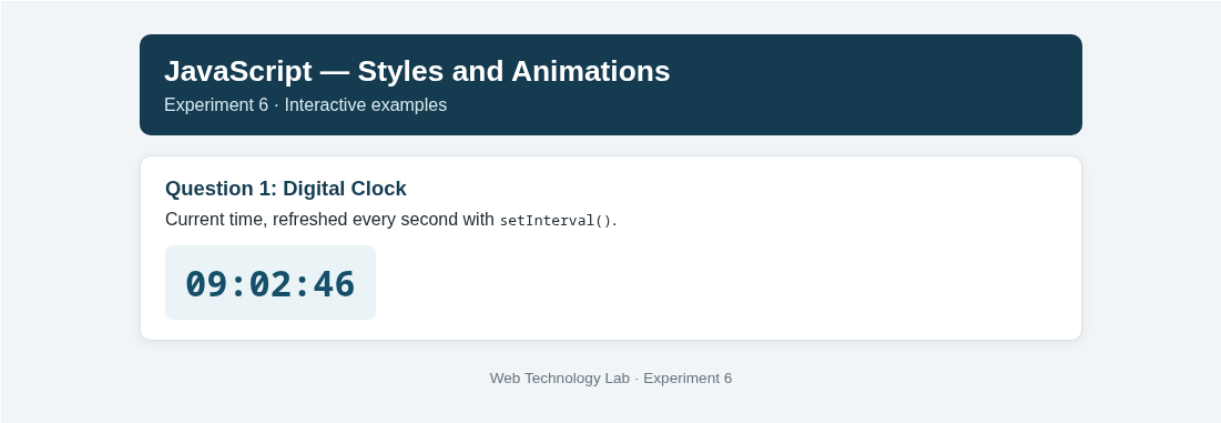
Question 1: Digital Clock

Program code:

```
HTML:
<output id="clock" aria-live="polite"></output>

JavaScript:
function displayTime() {
  const now = new Date();
  const pad = value => String(value).padStart(2, "0");
  document.getElementById("clock").textContent =
    `${pad(now.getHours())}:${pad(now.getMinutes())}:${pad(now.getSeconds())}`;
}
displayTime();
window.setInterval(displayTime, 1000);
```

Program Output:



Output 1: The program displays the result for digital clock.

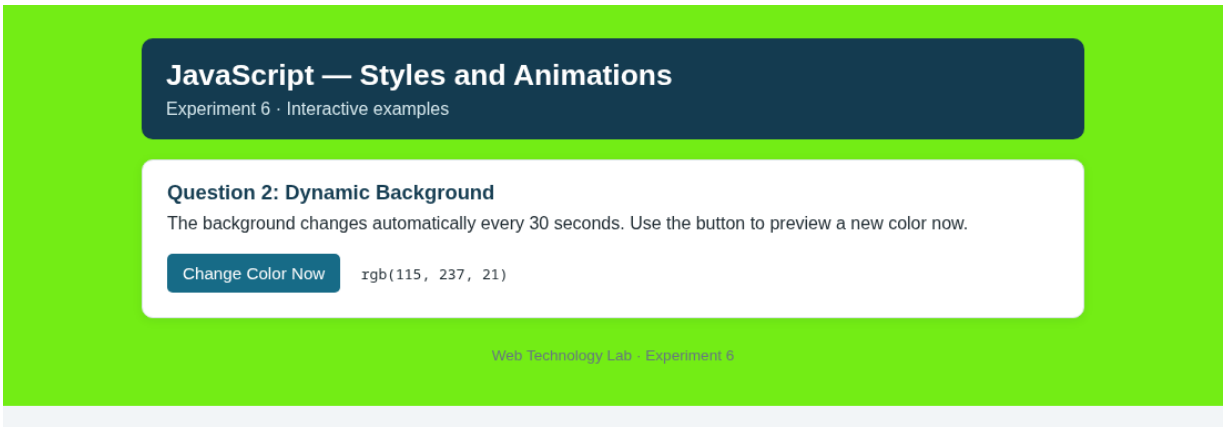
Question 2: Dynamic Background Color

Program code:

```
HTML:
<button id="changeColor">Change Color Now</button>
<span id="colorValue"></span>

JavaScript:
function setBackgroundColor() {
  const rgb = Array.from({ length: 3 }, () => Math.floor(Math.random() * 256));
  const color = `rgb(${rgb.join(", ")})`;
  document.body.style.backgroundColor = color;
  document.getElementById("colorValue").textContent = color;
}
setBackgroundColor();
window.setInterval(setBackgroundColor, 30000);
document.getElementById("changeColor").addEventListener("click", setBackgroundColor);
```

Program Output:



Output 2: The program displays the result for dynamic background color.

Question 3: Mouse Font Size

Program code:

```
HTML:
<p id="zoomText">Move the pointer over this paragraph to enlarge it.</p>

JavaScript:
const zoomText = document.getElementById("zoomText");
zoomText.addEventListener("mouseover", () => { zoomText.style.fontSize = "28px"; });
zoomText.addEventListener("mouseout", () => { zoomText.style.fontSize = "16px"; });
```

Program Output:



Output 3: The program displays the result for mouse font size.

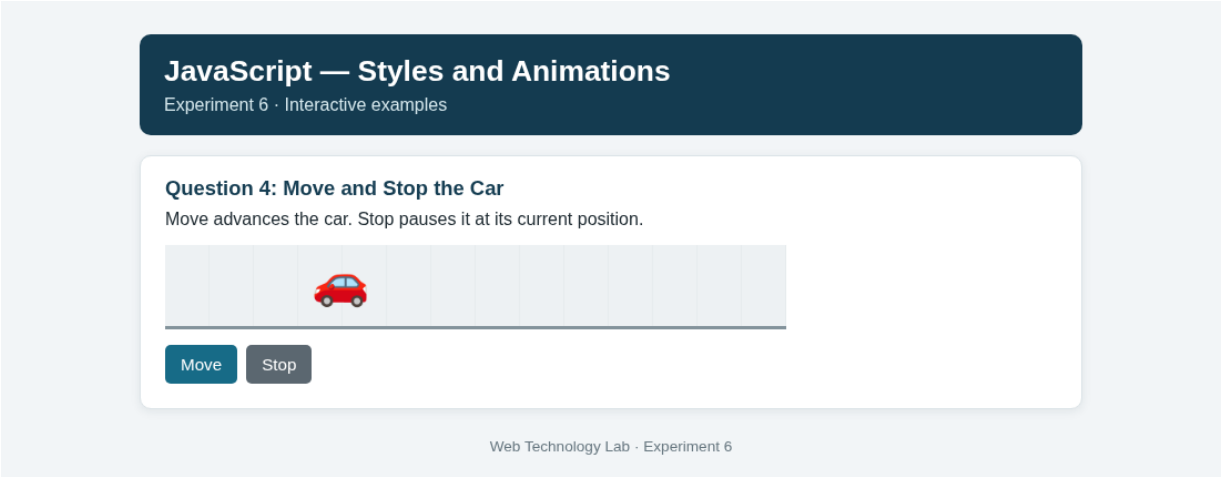
Question 4: Move and Stop the Car

Program code:

```
HTML and CSS:
<div id="track"><span id="car" role="img" aria-label="car">■</span></div>
<button id="moveCar">Move</button><button id="stopCar">Stop</button>
<style>
#track { position: relative; width: 560px; height: 76px; overflow: hidden; }
#car { position: absolute; left: 0; top: 12px; font-size: 42px; }
</style>

JavaScript:
const car = document.getElementById("car");
const track = document.getElementById("track");
let animationTimer = null;
function moveRight() {
  const maxLeft = track.clientWidth - car.offsetWidth;
  const nextLeft = car.offsetLeft + 2;
  if (nextLeft >= maxLeft) { car.style.left = `${maxLeft}px`; animationTimer = null; return; }
  car.style.left = `${nextLeft}px`;
  animationTimer = window.setTimeout(moveRight, 20);
}
function startCar() { if (animationTimer === null) moveRight(); }
function stopCar() {
  if (animationTimer !== null) window.clearTimeout(animationTimer);
  animationTimer = null;
}
document.getElementById("moveCar").addEventListener("click", startCar);
document.getElementById("stopCar").addEventListener("click", stopCar);
```

Program Output:



Output 4: The program displays the result for move and stop the car.

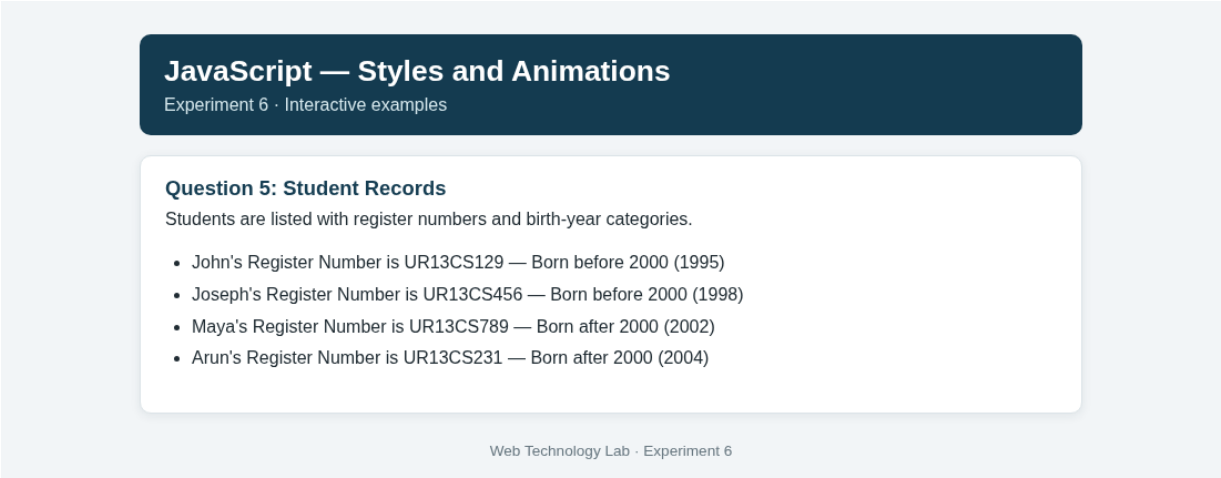
Question 5: Student Records and Birth-Year Category

Program code:

```
HTML:
<ul id="studentList"></ul>

JavaScript:
class Student {
  constructor(name, regno, yearOfBirth) {
    this.name = name; this.regno = regno; this.yearOfBirth = yearOfBirth;
  }
}
const students = [
  new Student("John", "UR13CS129", 1995), new Student("Joseph", "UR13CS456", 1998),
  new Student("Maya", "UR13CS789", 2002), new Student("Arun", "UR13CS231", 2004)
];
const studentList = document.getElementById("studentList");
students.forEach(student => {
  const category = student.yearOfBirth < 2000 ? "Born before 2000" : "Born after 2000";
  const item = document.createElement("li");
  item.textContent = `${student.name}'s Register Number is ${student.regno} – ${category}`;
  studentList.appendChild(item);
});
```

Program Output:



Output 5: The program displays the result for student records and birth-year category.

Result

All five JavaScript programs were implemented successfully. Their outputs were captured from the running Experiment 6 webpage and included above.